

Learned Copper: Guiding PCB Autorouting with a Model Trained on Human Routed Boards

CS57600 Machine Learning, Fall 2026. Term Project Proposal.

Team Information

Om Singhal. I am working on this project alone.

Background and Motivation

Routing is the part of printed circuit board design that still consumes the most engineering time. Once the parts are placed, someone has to draw every copper trace (a path joining pads of the same net) and via (a plated hole carrying a trace between layers) by hand, and the automatic routers engineers bolt onto tools such as KiCad, usually Freerouting, are known for layouts that get thrown away. Two things changed recently. Commercial systems from InstaDeep and Quilter showed that reinforcement learning can route boards, and in 2026 LG AI Research published PCBWorld [1], an open benchmark that scores agents on real KiCad boards with KiCad's own design rule checker. Its best agent beats Freerouting on small boards (86 percent passed cleanly against 80) but passes fewer than half of the medium ones, where Freerouting passes 78 percent, and the largest boards were left untested as an open challenge. Meanwhile thousands of boards routed by real engineers sit in public open hardware repositories. PCBench packaged some of them for routing research and PCBWorld evaluates on 679 of those, but as far as I can tell nobody has trained a model on the copper itself at the scale these collections allow. I work on PCB design software, so I have a practical interest in what such a model can learn.

Problem / Question

Can a model trained on human routed boards predict where copper belongs well enough to make a conventional router better? Given a board with placed footprints (component outlines with their pads), a netlist (which pads must be connected) and design rules (minimum widths and clearances) but no traces: (1) treating this as per pixel classification, that is image segmentation, how accurately can a model predict the copper each layer will carry, (2) when that prediction steers a router as a cost map, does the router complete more boards with shorter traces and fewer vias than on its own, and (3) how much does a careless evaluation overstate the answer, given that open hardware collections are full of forks and revisions of the same board? I expect the third question to matter as much as the first two. The project starts with two layer boards, which make up most open hardware designs, and extends to four layers only if time permits.

Dataset(s)

Primary candidate: the collection published on Hugging Face as bshada/open-schematics [2], licensed CC BY 4.0. It holds 87,931 records of mostly KiCad projects gathered from public repositories, with some Altium files mixed in. Each record is one schematic file plus a list of every PCB layout file in the same project, the component list and the source repository. The list can be empty, and records from one project repeat the same boards, so the number of distinct routed layouts is unknown until the set is scanned: a sample from the start had layouts in every record, while records near the end had none. It suits the project because the layouts hold routing decisions by real engineers, the compilation license allows a derived dataset with attribution (the boards inside carry their own licenses, which the derived set will record), and the files are plain text that KiCad's pcbnew Python module loads directly. Preparation keeps boards that are fully connected and pass the design rule check, removes duplicates

and forks, caps the board size, then strips the tracks and vias to form the problem while the original copper stays as the answer.

Alternatives under consideration: the 679 boards packaged with PCBWorld, which have fixed splits and published baselines for 99 small and 10 medium boards and so serve mainly for evaluation, and the full PCBench pool on GitHub [3], 1,182 boards converted to KiCad 9, as a backup if the primary collection yields too few fully routed boards.

Initial Approach

Each board is rasterized into a multichannel image: outline, pads and keepouts per layer, and an encoding of which pads must be joined (an early design decision, since the number of nets varies). A convolutional encoder decoder network in the UNet style predicts copper occupancy per layer, scored against the human copper by intersection over union on trace pixels. Gradient boosting on hand built local features, such as the distance to the nearest pad of the same net and the local pad density, is trained on the same targets so that the neural model has to earn its place. The prediction then guides a router. Neither Freerouting nor PCBWorld accepts an external cost map, so the primary mechanism is a small grid based maze router written for this project that takes the prediction as a per cell cost; steering Freerouting through its open source or its exposed settings is a stretch goal. Guided and unguided routing are compared on held out boards by completion rate, wire length, via count and design rule violations, all measured by KiCad. Splits are by repository owner, and the experiment is repeated with a random split to measure how much near duplicate boards inflate the numbers. Training uses free Kaggle or Colab GPUs on a capped subset first, for example 2,000 boards at 256 by 256 pixels; parsing, rule checking and routing run on a laptop CPU. If time permits, the cleaned dataset and the model will be released on Hugging Face.

Initial Project Plan

The plan is tentative and will be adjusted as the project progresses.

- Weeks 1 to 2: parsing and rasterization pipeline on about 200 boards, Freerouting run on them, baseline completion rate recorded. This step decides whether the idea is worth pursuing. If the parsing yield is too low or batch routing fails, the project narrows to copper prediction on the PCBWorld boards.
- Weeks 3 to 4: full collection processed, filtered and deduplicated; data exploration (layer counts, board sizes, net counts, trace density, share of near duplicates); owner based splits frozen.
- Weeks 5 to 6: copper prediction model and gradient boosting baseline trained, errors studied, midway report written.
- Weeks 7 to 8: router guidance experiments, evaluated with KiCad's design rule checker.
- Week 9: random split comparison and error analysis.
- Weeks 10 to 11: presentation and final report, on the dates set by the course.

References

[1] PCBWorld: A Benchmark Environment for Engine Grounded PCB Design Automation, arXiv 2607.05915, github.com/LGAI-Research/PCBWorld. [2] huggingface.co/datasets/bshada/open-schematics. [3] github.com/PCBench/PCBench.